

EVI\$ION YB 강의세션 시스템 해킹 2주차

어셈블리,
프롤로그&에필로그,
gcc&gdb

컴퓨터는 어떤 방식으로 프로그램을 이해하고 실행할까?

시작하기 전에...

- 오늘은 실습이 있어요.
- 가상머신이 잘 실행되는지 확인해주세요!
 - 인터넷 연결 확인 : firefox 클릭했을 때 실행되어야 함.

목차

- 어셈블리
- 프롤로그와 에필로그
- gcc와 gdb
- 실습
- 과제 안내

어셈블리

우리가 쓴 코드를 컴퓨터는 어떻게 실행할까?

컴퓨터가 소스코드를 실행하려면...

- Step1. C 소스코드 작성
- Step2. 전처리 : 전처리기 → 확장된 C 소스코드
- Step3. 컴파일 : 컴파일러 → 어셈블리 코드
- Step4. 어셈블 : 어셈블러 → 목적 프로그램
- Step5. 링킹 : 외부 함수들을 라이브러리에서 찾아 연결
- Step6. 로딩 : 실행파일을 메모리에 올리고 프로그램 시작

어셈블리

- 컴퓨터가 소스코드를 해석하며 등장한 그것
- 기계어와 1:1 매핑
- 메모리 구조가 선명
- 실제 CPU 동작 추적 가능
- 마침! 역분석(디스어셈블) 가능

어셈블리어 표기 방법

우리가 볼 건 인텔기반 표기 방법!

	인텔	AT&T
레지스터 표현	eax	%eax
값의 표현	AABBCCDDh 또는 0xAABBCCDD	\$0xAABBCCDD
명령어 크기 표현	mov	movl (long형) movb(byte형)
메모리 주소	[주소]	(주소)
오퍼랜드 방향	목적지(dst) → 소스(src)	소스(src) → 목적지(dst)

[출처] 『리버싱과 시스템 해킹의 원리』 (김민수 저, 인피니티북스, 2020), 68p 표

어셈블리 명령어(이건 알고 가요)

- mov/lea
- add/sub
- push/pop
- call/ret
- cmp/je/jne

mov dst, src

```
mov eax, ebx      ; 레지스터 값 → dst  
mov eax, 0x42     ; 특정 값 → dst  
mov eax, [ebp-0x4] ; 특정 메모리 저장 값 → dst
```

- dst에 src의 값을 복사
- 레지스터에 저장된 값 복사
- 특정 값 복사
- 특정 메모리에 저장된 값 복사

lea dst, src

```
lea eax, [ebp-0x4]      ; ebp-0x4을 eax에 저장  
mov eax, [ebp-0x4]      ; ebp-0x4에 저장된 값을 eax에 저장
```

- Load Effective Address의 약자
- 메모리 "주소" 를 dst에 넣음
- 실제 메모리 접근 X
- 포인터 연산, 오프셋 계산에 쓰임

add 대상, 가산값 / sub 대상, 감산값

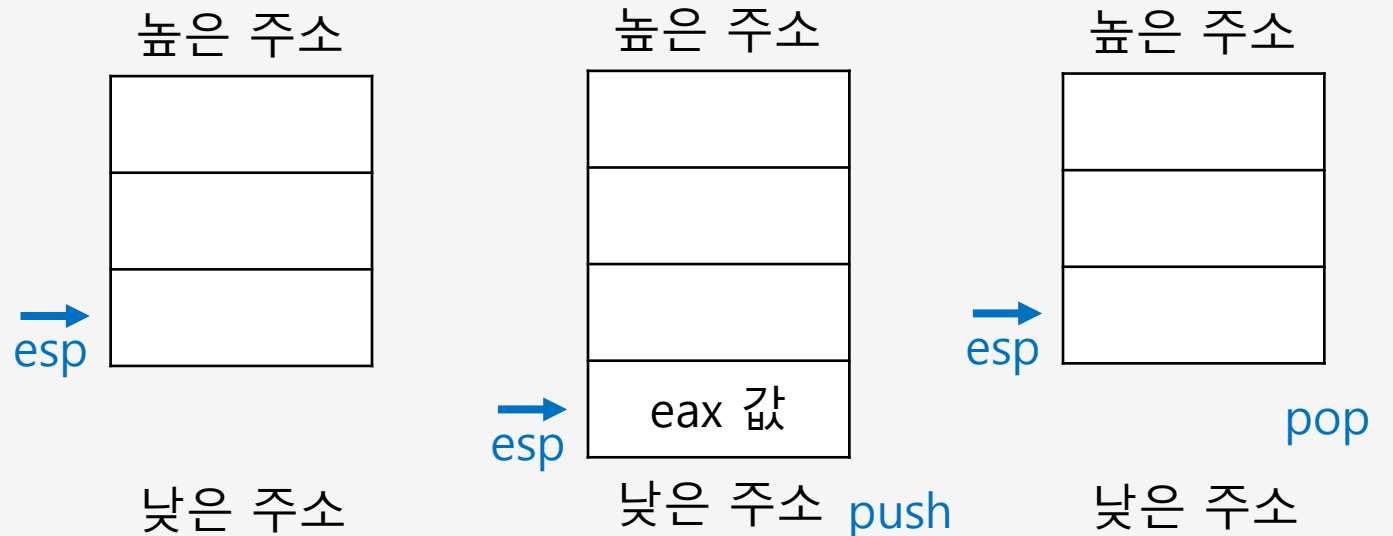
```
add eax, ebx          ; eax = eax + ebx
sub esp, 0x10          ; esp = esp - 0x10(16)
add [ebp-0x4], [ebp-0x8] ; 불가능!
```

- add : 더하기
- sub : 빼기
- 주의! 메모리랑 메모리 가져다 더하기/빼기 불가능
더하기 빼기는 항상 레지스터 사용

push 대상 / pop 대상

push eax ; 스택 맨 위에 eax값을 넣음(esp 감소 = 스택 증가)
pop eax ; 스택 맨 위 값을 꺼내서 eax에 넣음(esp 증가 = 스택 감소)

- push : 스택 맨 위에 값 추가
→ 스택 증가 = esp 감소
- pop : 스택 맨 위 값 꺼내기
→ 스택 감소 = esp 증가



cmp 대상1, 대상2 / [je/jne/jmp] LABEL

```
cmp A, B      ; A와 B 비교 → 플래그 기록  
je LABEL      ; 같으면 LABEL로 점프 : if (A == B)  
jne LABEL     ; 다르면 LABEL로 점프 : if (A != B)  
jmp LABEL     ; 조건 없이 LABEL로 점프
```

- cmp A, B : $A - B$ 연산 수행 → 0이면 ZF(플래그 레지스터) 켜기(1)
- je : ZF가 켜져 있으면(1) → LABEL로 점프
- jne : ZF가 꺼져 있으면(0) → LABEL로 점프
- jmp LABEL : 조건 없이 LABEL로 점프

call 호출함수 / ret

```
call 함수주소      ; 해당 주소의 함수 호출  
ret                ; 리턴 주소로 점프
```

- call : 리턴 주소 push → 함수 호출
 - push(리턴 주소) + jump(호출한 함수로)
- ret : 리턴 주소 pop(eip에 저장) = 리턴 주소로 점프
 - pop(리턴 주소를 eip로) + jump(eip에 저장된 값으로)

보너스! 스택 정렬(with. and)

```
and esp, 0xffffffff0
```

- 스택 정렬(Stack alignment)
 - : 스택 포인터(esp)를 특정 배수(보통 16바이트)의 주소로 맞추는 작업
 - eg) 정렬 전: esp = 0x35 (16 배수 아님)
정렬 후: esp = 0x30 (16 배수로 맞춤)
- 강제인가? 32bit에서는 필수 X, but 컴파일러가 종종 맞춰줌

퀴즈1. 다음 어셈블리 코드 중 수행 결과로 eax에 3이 저장되는 것은?

1
mov eax, 3
mov ebx, 2
sub ebx, 1
add eax, ebx
mov eax, ebx

2
mov ebx, 5
mov ecx, 2
sub ebx, ecx
sub ecx, 1
mov eax, ebx

3
mov ebx, 4
mov ecx, 2
sub ecx, ebx
add ebx, 1
mov eax, ebx

4
mov eax, 4
mov ebx, 1
mov ecx, ebx
sub ecx, eax
mov eax, ecx

퀴즈1. 다음 어셈블리 코드 중 수행 결과로 eax에 3이 저장되는 것은?

1

```
mov eax, 3    ;eax = 3
mov ebx, 2    ;ebx = 2
sub ebx, 1    ;ebx = 1
add eax, ebx  ;eax = 4
mov eax, ebx  ;eax = 1
```

3

```
mov ebx, 4    ;ebx = 4
mov ecx, 2    ;ecx = 2
sub ecx, ebx  ;ecx = -2
add ebx, 1    ;ebx = 5
mov eax, ebx  ;eax = 5
```

2

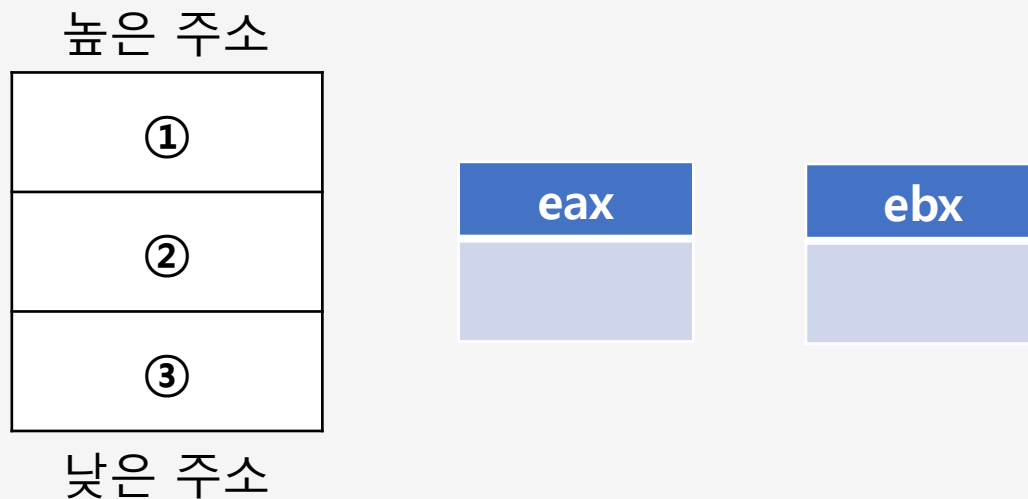
```
mov ebx, 5    ;ebx = 5
mov ecx, 2    ;ecx = 2
sub ebx, ecx  ;ebx = 3
sub ecx, 1    ;ecx = 1
mov eax, ebx  ;eax = 3
```

4

```
mov eax, 4    ;eax = 4
mov ebx, 1    ;ebx = 1
mov ecx, ebx  ;ecx = 1
sub ecx, eax  ;ecx = -3
mov eax, ecx  ;eax = -3
```

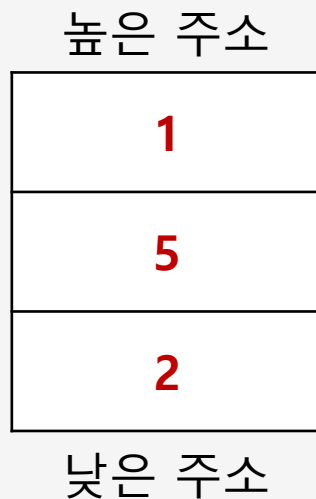
퀴즈2. 다음 어셈블리 코드 수행 결과 eax, ebx에 저장되는 값과 스택에 저장되는 있는 값은? (스택은 높은 주소에서 낮은 주소로 쌓인다)

```
push 1
mov eax, 2
mov ebx, eax
add ebx, 3
push ebx
push eax
push 4
pop ebx
```



퀴즈2. 다음 어셈블리 코드 수행 결과 eax, ebx에 저장되는 값과 스택에 저장되는 있는 값은? (스택은 높은 주소에서 낮은 주소로 쌓인다)

```
push 1      ;[1]
mov eax, 2   ;eax = 2
mov ebx, eax ;ebx = 2
add ebx, 3   ;ebx = 5
push ebx     ;[1, 5]
push eax     ;[1, 5, 2]
push 4       ;[1, 5, 2, 4]
pop ebx      ;[1, 5, 2]
```



정리

- mov/lea : 값 복사 / 주소 복사
- add/sub : 더하기 / 빼기
- push/pop : 스택에 값 넣기 / 스택에서 값 꺼내기
- call/ret : 함수 부르기 / 함수 떠나기
- cmp/je/jne : 비교 / 조건 점프 (if (A == B){..})

프롤로그와 에필로그

함수 시작하고 끝내기

함수 시작 : 프롤로그(Prologue)

```
push ebp      ; 이전 ebp 백업  
mov ebp, esp  ; ebp를 현재 스택프레임 기준으로  
sub esp, 0x10 ; 로컬 변수 공간 확보
```

- 목적 : 새로운 스택 프레임 새로 만들기
 1. 이전 ebp 백업
 2. ebp를 현재 스택프레임 기준으로 변경
 3. 로컬 변수 공간 확보

함수 시작 : 프롤로그(Prologue)

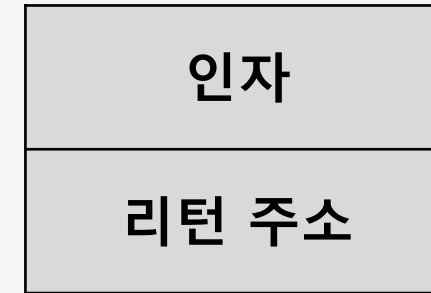
caller

→ push argv
call function
: push retrun address
callee로 이동

callee

push ebp
mov ebp, esp
sub esp, 0x10

높은 주소



낮은 주소

함수 시작 : 프롤로그(Prologue)

caller

```
push argv  
call function  
: push retrun address  
callee로 이동
```

callee

```
→ push ebp  
mov ebp, esp  
sub esp, 0x10
```



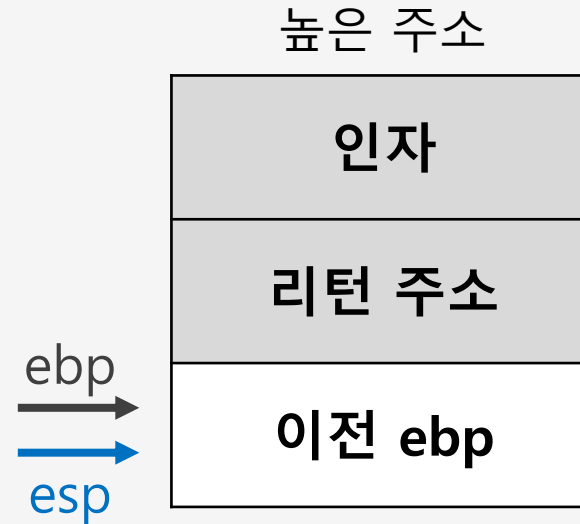
함수 시작 : 프롤로그(Prologue)

caller

```
push argv  
call function  
: push retrun address  
callee로 이동
```

callee

```
→ push ebp  
mov ebp, esp  
sub esp, 0x10
```



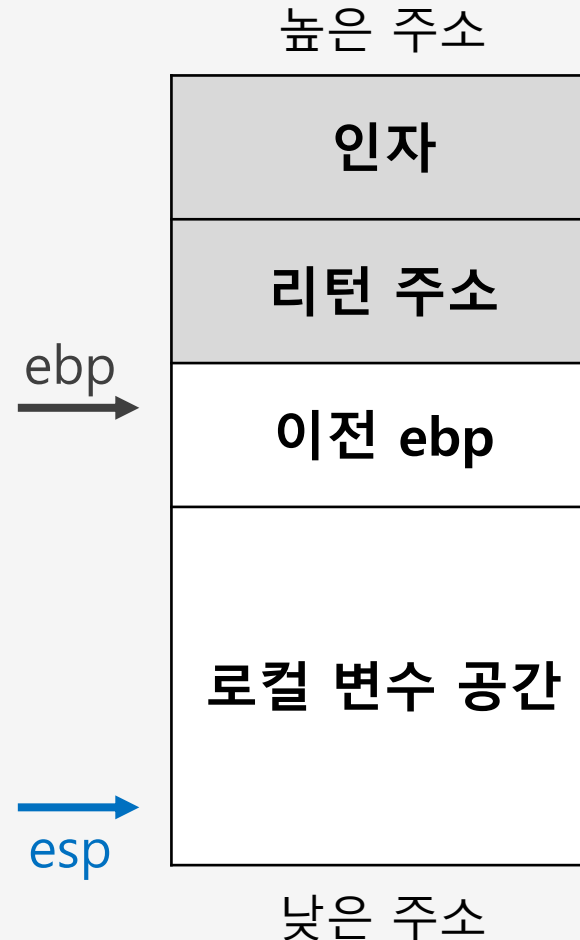
함수 시작 : 프롤로그(Prologue)

caller

```
push argv  
call function  
: push retrun address  
callee로 이동
```

callee

```
→ push ebp  
mov ebp, esp  
→ sub esp, 0x10
```



함수 종료 : 에필로그(Epilogue)

```
leave      ; mov esp, ebp + pop ebp  
ret        ; pop eip, jmp eip
```

- 목적 : 스택 프레임 정리하기
 1. (leave) mov esp, ebp : 로컬 변수 공간 제거
 2. (leave) pop ebp : 이전 ebp로 ebp 복원 (caller의 ebp 복원)
 3. (ret) pop eip : 스택에서 리턴 주소를 꺼내 eip에 넣기
 4. (ret) jmp eip : 리턴 주소로 복귀 (caller로 점프)

함수 종료 : 에필로그(Epilogue)

callee



leave

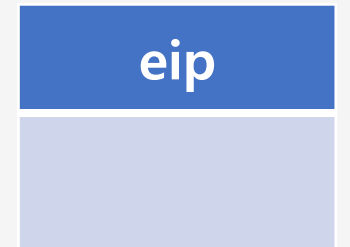
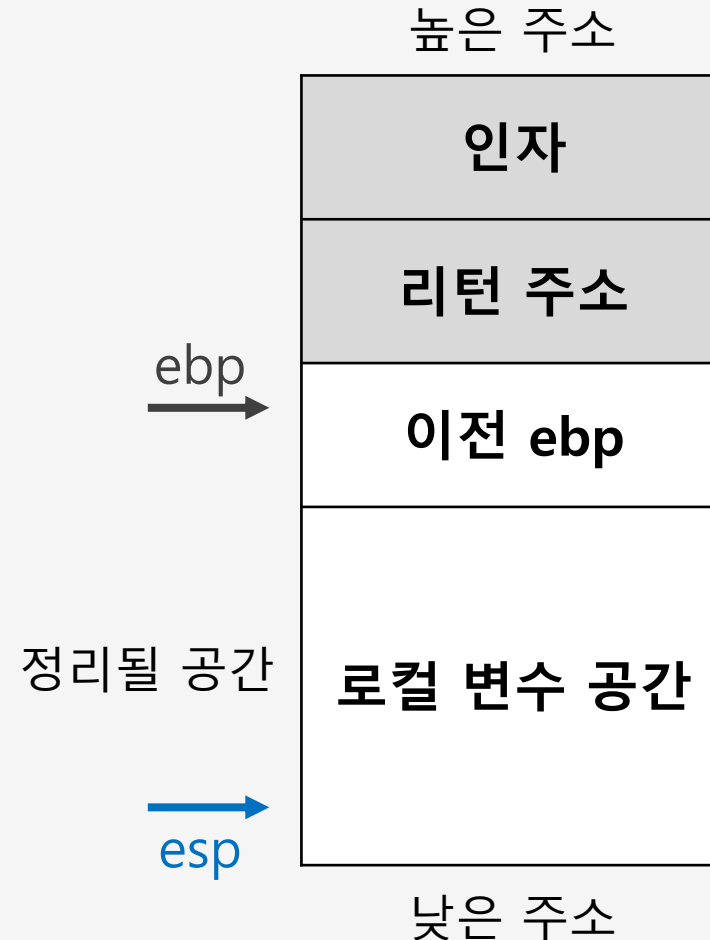
```
: mov esp, ebp  
: pop ebp
```

ret

```
: pop eip  
: jmp eip
```

caller

```
add esp, 0x4
```



함수 종료 : 에필로그(Epilogue)

callee

leave

: mov esp, ebp
pop ebp

ret

: pop eip
jmp eip

caller

add esp, 0x4

높은 주소



ebp
esp

낮은 주소

eip

함수 종료 : 에필로그(Epilogue)

callee

leave

: mov esp, ebp
pop ebp

ret

: pop eip
jmp eip

caller

add esp, 0x4

높은 주소

인자

리턴 주소

ebp는 caller의
베이스를
가르키는 중

eip

낮은 주소

함수 종료 : 에필로그(Epilogue)

callee

leave

: mov esp, ebp
pop ebp

ret

: pop eip
jmp eip



→
esp

높은 주소

인자

caller

add esp, 0x4

eip

리턴 주소

낮은 주소

함수 종료 : 에필로그(Epilogue)

callee

leave

: mov esp, ebp
pop ebp

ret

: pop eip
jmp eip



높은 주소

인자

eip에 저장된 주소로 점프

caller

add esp, 0x4

eip

리턴 주소

낮은 주소

함수 종료 : 에필로그(Epilogue)

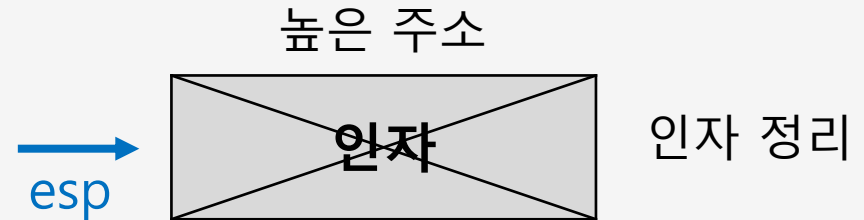
callee

leave

```
: mov esp, ebp  
: pop ebp
```

ret

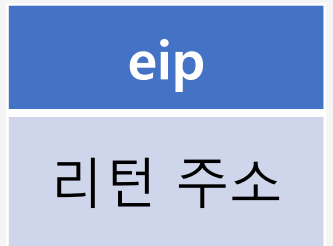
```
: pop eip  
: jmp eip
```



eip에 저장된 주소로 점프

caller

→ **add** esp, 0x4



낮은 주소

프로로그/에필로그

- 프로로그 : callee가 자기 스택프레임 만들기

```
push ebp      ; 이전 ebp 백업  
mov ebp, esp  ; ebp를 현재 스택프레임 기준으로  
sub esp, 0x10 ; 로컬 변수 공간 확보
```

- 에필로그 : callee가 자기 스택프레임 치우기

```
leave         ; mov esp, ebp + pop ebp  
ret           ; pop eip, jmp eip
```

gcc, gdb

소스코드 실행시키고, 컴퓨터가 코드를 실행하는 과정 따라가보기

gcc

- GNU Compiler Collection의 약자
- GNU 프로젝트의 오픈 소스 컴파일러 (CLI 기반)
- C/C++ 등 언어를 컴파일 가능
- 기본 사용법

```
gcc -[옵션] -o [출력파일] [소스파일]
```

gdb

- GNU 프로젝트의 디버거 (CLI 기반)
- 응용 프로그램을 디버깅하는데에 사용됨
 - 프로그램 실행 흐름/변수/레지스터 확인 가능
- C/C++ 등 다양한 언어 지원
- 디버깅 심볼이 필요 시 gcc 컴파일에 -g 옵션 사용
- 디버깅에 필요한 여러 기능 제공

gdb

- 기본 사용법

```
gdb [프로그램명]  
(gdb) [명령어]
```

gdb 명령어

명령어	설명	사용 예시
run / r	프로그램 실행	(gdb) run
break / b [함수명]	해당 함수명 시작점에 브레이크포인트 설정	(gdb) break main
next / n	다음 줄로 이동 (함수 내부 안 들어감)	(gdb) next
step / s	다음 줄 실행 (함수 내부로 들어감)	(gdb) step
continue	다음 브레이크포인트까지 실행	(gdb) continue

gdb 명령어

명령어	설명	사용 예시
print / p [변수명]	변수 값 보기	(gdb) print x
info registers	레지스터 전체 보기	(gdb) info registers
disas / disassemble [함수명]	디스어셈블링, 어셈블리 보기	(gdb) disas main
x/[범위][출력형식] [함수명/주소]	메모리 상태 검사	(gdb) x/20x \$esp

실습

소스코드 실행시키고, 컴퓨터가 코드를 실행하는 과정 직접 추적해보기

같이 해보기

1. 가상머신 환경 세팅 : gcc, gdb, 32bit 실행 환경
2. 소스코드 작성 (제가 해옴)
3. gcc로 소스코드 컴파일
4. gdb로 디스어셈블링
5. 어셈블리 보고 스택 프레임 추측
6. 동적 분석으로 스택 확인

⚠ 컴퓨터마다 스택 프레임 약간씩 다르게 나올 수 있음

1. 가상머신 환경 세팅

1. gcc, gdb, 32bit 실행환경 설치(한줄씩 치고 엔터)

```
sudo apt update  
sudo dpkg --add-architecture i386  
sudo apt update  
sudo apt install -y libc6:i386  
sudo apt install -y gcc-multilib  
sudo apt install -y gdb-multiarch
```

password 입력하라고 하면

가상머신 만들 때 입력한 그 password 입력
(입력해도 화면에 안 보이는건 정상)

2. gcc, gdb 설치 확인

```
gcc --version  
gdb --version
```

⚠ 복붙이 안돼요 → 아래 명령어 입력

```
sudo apt update
```

```
sudo apt install -y open-vm-tools-desktop
```

```
sudo reboot
```

2. 소스코드 작성

1. 실습용 디렉터리 만들고 이동

```
mkdir work  
cd work
```

2. 텍스트 편집기 열고 우측 소스코드 입력

```
nano test.c
```

⚠ nano 없대요 → 설치

```
sudo apt install -y nano
```

```
// filename: test.c  
#include<stdio.h>  
  
int sum(int x, int y){  
    int result = x + y;  
    return result;  
}  
  
int main(){  
    int a = 2, b = 3, c;  
    c = sum(a, b);  
    printf("%d\n", c);  
    return 0;  
}
```

GNU nano 6.2

```
//filename: test.c
```

```
#include<stdio.h>
```

```
int sum(int x, int y){  
    int result = x + y;  
    return result;  
}
```

```
int main(){  
    int a = 2, b = 3, c;  
    c = sum(a, b);  
    printf("%d\n", c);  
    return 0;  
}
```



root@b497b



```
root@b497bac1368d:/# mkdir /work
```

```
root@b497bac1368d:/# cd /work
```

```
root@b497bac1368d:/work# nano test.c
```

```
root@b497bac1368d:/work# ls
```



```
test.c
```

```
root@b497bac1368d:/work# cat test.c
```



```
//filename: test.c
```

```
#include<stdio.h>
```



```
int sum(int x, int y){  
    int result = x + y;  
    return result;  
}
```



```
int main(){  
    int a = 2, b = 3, c;  
    c = sum(a, b);  
    printf("%d\n", c);  
    return 0;  
}
```



```
root@b497bac1368d:/work#
```



ls, cat으로 test.c 가 잘 저장되는지 확인

3. gcc로 컴파일

아래 명령어 입력해서 컴파일

```
gcc -m32 -fno-stack-protector -fno-pie -no-pie -fno-omit-frame-pointer  
-mpreferred-stack-boundary=2 -O0 -g -o test test.c
```

⚠ file 없대요 → 설치

./test 입력해서 실행해보기 + file test 쳐서 32bit 라고 나오는지 확인해보기

sudo apt install -y file

```
name@name-VMware-Virtual-Platform:~/Desktop/work$ gcc -m32 -fno-stack-protector -fno-pie -no-pie -fno-omit-fr  
ame-pointer -mpreferred-stack-boundary=2 -O0 -g -o test test.c  
name@name-VMware-Virtual-Platform:~/Desktop/work$ ls  
test  test.c  
name@name-VMware-Virtual-Platform:~/Desktop/work$ ./test  
5  
name@name-VMware-Virtual-Platform:~/Desktop/work$ file test  
test: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux  
.so.2, BuildID[sha1]=3f907db9931f59174cb6bc32e318fc61b99a28e7, for GNU/Linux 3.2.0, with debug_info, not stri  
pped  
name@name-VMware-Virtual-Platform:~/Desktop/work$
```

4. gdb로 디스어셈블링

1. gdb 실행

```
gdb test
```

2. intel 문법으로 설정

```
set disassembly-flavor intel  
set pagination off
```

```
name@name-VMware-Virtual-Platform:~/Desktop/work$ gdb test  
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-git  
Copyright (C) 2024 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
This is free software: you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
Type "show copying" and "show warranty" for details.  
This GDB was configured as "x86_64-linux-gnu".  
Type "show configuration" for configuration details.  
For bug reporting instructions, please see:  
<https://www.gnu.org/software/gdb/bugs/>.  
Find the GDB manual and other documentation resources online at:  
  <http://www.gnu.org/software/gdb/documentation/>.  
  
For help, type "help".  
Type "apropos word" to search for commands related to "word"...  
Reading symbols from test...  
(gdb) set disassembly-flavor intel  
(gdb) set pagination off  
(gdb)
```

4. gdb로 디스어셈블링

⚠ 디스어셈블링 결과는
컴퓨터 마다 다를 수 있음

1. main 디스어셈블링

```
disas main
```

2. sum 디스어셈블링

```
disas sum
```

```
(gdb) disas main
Dump of assembler code for function main:
   0x0804917c <+0>:    push    ebp
   0x0804917d <+1>:    mov     ebp,esp
   0x0804917f <+3>:    sub     esp,0xc
   0x08049182 <+6>:    mov     DWORD PTR [ebp-0x4],0x2
   0x08049189 <+13>:   mov     DWORD PTR [ebp-0x8],0x3
   0x08049190 <+20>:   push    DWORD PTR [ebp-0x8]
   0x08049193 <+23>:   push    DWORD PTR [ebp-0x4]
   0x08049196 <+26>:   call    0x08049166 <sum>
   0x0804919b <+31>:   add     esp,0x8
   0x0804919e <+34>:   mov     DWORD PTR [ebp-0xc],eax
   0x080491a1 <+37>:   push    DWORD PTR [ebp-0xc]
   0x080491a4 <+40>:   push    0x804a008
   0x080491a9 <+45>:   call    0x08049040 <printf@plt>
   0x080491ae <+50>:   add     esp,0x8
   0x080491b1 <+53>:   mov     eax,0x0
   0x080491b6 <+58>:   leave
   0x080491b7 <+59>:   ret
End of assembler dump.
(gdb) █
```


4. gdb로 디스어셈블링

⚠ 디스어셈블링 결과는
컴퓨터 마다 다를 수 있음

1. main 디스어셈블링

```
disas main
```

2. sum 디스어셈블링

```
disas sum
```

```
(gdb) disas sum
Dump of assembler code for function sum:
0x08049166 <+0>:      push    ebp
0x08049167 <+1>:      mov     ebp,esp
0x08049169 <+3>:      sub     esp,0x4
0x0804916c <+6>:      mov     edx,DWORD PTR [ebp+0x8]
0x0804916f <+9>:      mov     eax,DWORD PTR [ebp+0xc]
0x08049172 <+12>:     add     eax,edx
0x08049174 <+14>:     mov     DWORD PTR [ebp-0x4],eax
0x08049177 <+17>:     mov     eax,DWORD PTR [ebp-0x4]
0x0804917a <+20>:     leave
0x0804917b <+21>:     ret
End of assembler dump.
(gdb) █
```

5. 스택 프레임 추측

<main 스택 프레임 추측>



<sum 스택 프레임 추측>



5. 스택 프레임 추측 : main

1. 프로로그 확인
2. 변수 a, b에 값 저장하는 부분 확인
3. sum함수 호출 전 인자 b, a 순서로 전달하는 부분 확인
4. call sum

```
// filename: test.c
#include<stdio.h>

int sum(int x, int y){
    int result = x + y;
    return result;
}

int main(){
    int a = 2, b = 3, c;
    c = sum(a, b);
    printf("%d\n", c);
    return 0;
}
```

5. 스택 프레임 추측 : sum

1. 프롤로그 확인
2. result에 값 저장하는 부분 확인
3. 에필로그 확인

```
// filename: test.c
#include<stdio.h>

int sum(int x, int y){
    int result = x + y;
    return result;
}

int main(){
    int a = 2, b = 3, c;
    c = sum(a, b);
    printf("%d\n", c);
    return 0;
}
```

5. 스택 프레임 추측 : main

5. c에 값 저장되는 부분 확인
6. printf함수 호출 전 인자 변수 c, 문자열 주소
순으로 전달하는 부분 확인
→ x/s [주소] 로 확인 가능
7. call printf
8. 에필로그 확인

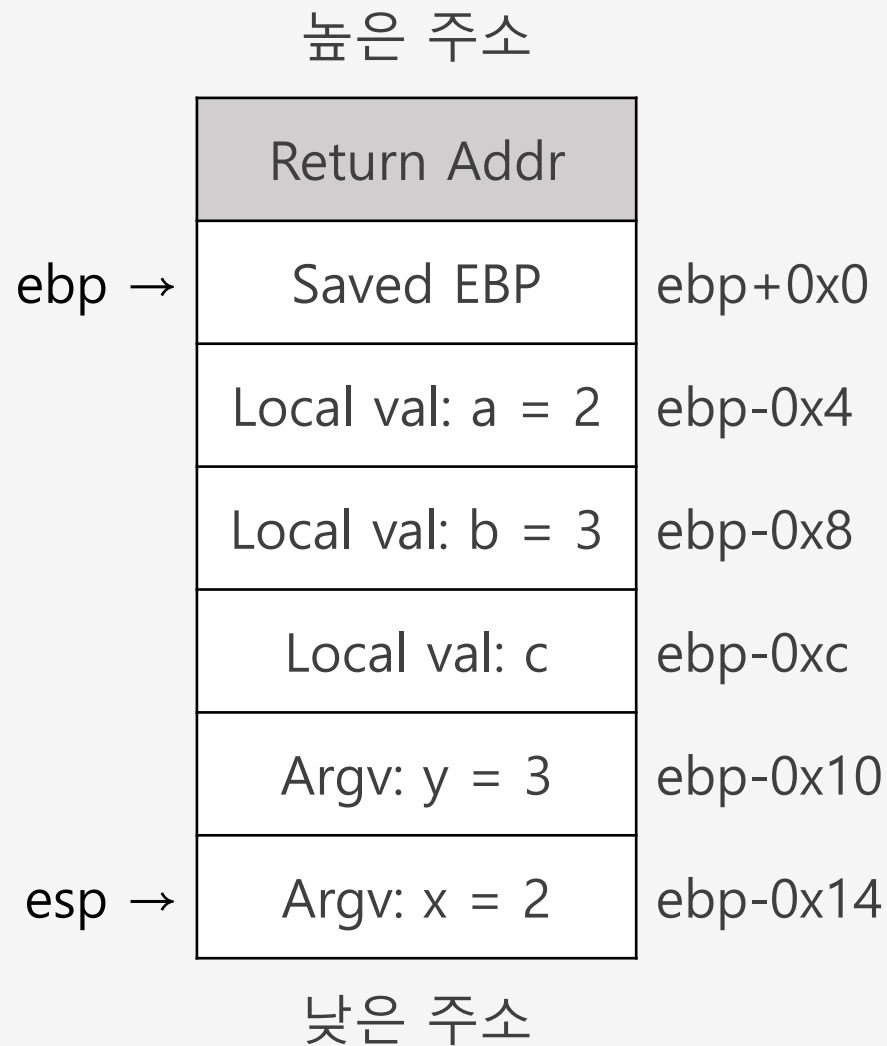
```
// filename: test.c
#include<stdio.h>

int sum(int x, int y){
    int result = x + y;
    return result;
}

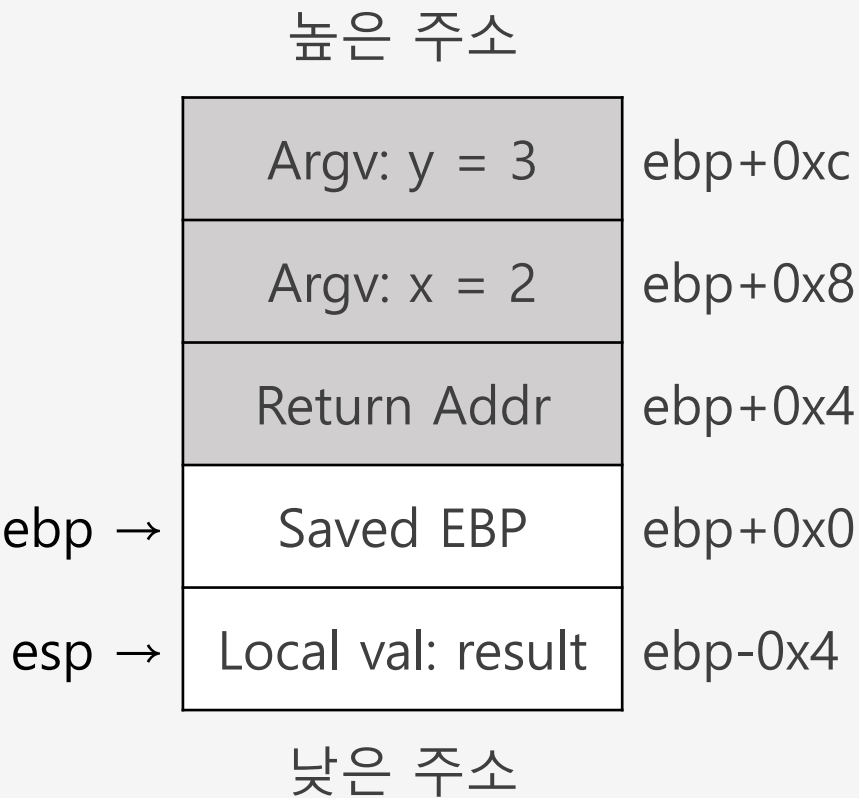
int main(){
    int a = 2, b = 3, c;
    c = sum(a, b);
    printf("%d\\n", c);
    return 0;
}
```

5. 스택 프레임 추측(예시)

<main의 sum 호출 직전 스택 프레임>



<main의 sum 호출 후 스택 프레임>



6. 동적 분석으로 스택 확인(간단하게)

1. break main : main 진입 직전에 브레이크 포인트 설정
2. run : 브레이크 포인트 직전까지만 실행
3. next : 다음 행 실행
4. disas main : 어디까지 진행됐는지 확인 가능
5. info register : 전체 레지스터 출력
6. x/4x \$esp : esp 위로 4칸 확인

과제 소개

복습용 과제

과제: gdb 디스어셈블링 + 스택 추적

과제 예시!(포맷 아님)

- gdb에서 디스어셈블 직접 실행
- 스택 프레임 그림 첨부
- 어셈블리에서 아래 항목 찾기
 - 프로로그, 변수 값 저장, 인자 push, 함수 호출, 에필로그
 - main/sum 각각에서 제시한 항목에 해당하는 어셈블리 코드 찾는 거

disas main 실행 스샷

1. 에필로그

```
push ebp  
mov ebp, esp  
sub esp, 0xc
```

....

스택 프레임 추정

스택 프레임 그림

제출 방식

- 블로그, 노션, 깃허브
- 공개글로 작성한 후 동아리 노션에 링크를 올려주세요!
- 기한 : 다음주 동아리 시작 전까지! (11/27 18:30까지)
 - 늦제출 : 다음날 자정 전까지 (11/28 23:59까지)

요약

- 어셈블리
 - mov, call, ret
- 프롤로그/에필로그
- gcc/gdb
 - run, break, info registers, next, x, disas...